

TeleAntiFraud 项目学习与复现手册

面向机器学习新手的音频+文本电诈检测项目说明

项目路径	C:\vscode_project\teledeceit_analysis
核心目标	用电话音频和文本提示训练 fraud/normal 二分类检测模型
推荐路线	冻结大模型编码器，缓存 embedding，再训练轻量融合分类头
硬件记录	训练日志记录为 RTX 3060 Laptop GPU 6GB；3060Ti 8GB 可沿用相同路线
生成日期	2026-05-19

提示

阅读建议：先看第 1-3 章理解项目，再按第 6 章命令复现。如果第一次接触深度学习，不要急着改模型，先把特征缓存、训练、预测完整跑一遍。

1. 这个项目在做什么

本项目使用 TeleAntiFraud 数据集训练一个电信诈骗检测模型。输入是一段电话音频，以及数据集中附带的文本提示；输出是二分类标签：normal 表示正常通话，fraud 表示诈骗通话。

对新手来说，最重要的是理解这个项目没有直接全量微调音频大模型。它先把大模型当作“特征提取器”：Whisper 负责把音频变成向量，中文 RoBERTa/BERT 负责把文本变成向量，然后训练一个很小的 MLP 分类器。这种做法显存压力小，适合 6GB/8GB 级别显卡。

你需要掌握的主线

- 数据解析：从 JSON 中取出 audio_path、text、answer。
- 特征缓存：把音频和文本转换成 embedding，并保存成 .pt 文件。
- 融合训练：拼接 audio embedding 与 text embedding，训练 MLP 分类头。
- 评估预测：计算 precision、recall、F1，并导出预测 CSV。

为什么不一开始做 SFT

SFT 数据有 2.7 万条，更适合训练“能解释原因的音频问答模型”。但这会占用更多显存和时间，还涉及 LoRA、量化、梯度累积等技巧。当前项目先做可复现的二分类基线，再把 SFT 作为进阶路线。

2. 数据集结构

项目读取的是 data 目录下已经解压的数据。README 给出的关键规模如下：

数据	路径	样本数	用途
二分类训练集	data/binary_classification/.../train.json	4000	训练 fraud/normal
二分类测试集	data/binary_classification/.../test.json	400	评估模型
SFT 训练集	data/sft/sft/train.jsonl	27146	后续音频指令微调
SFT 测试集	data/sft/sft/test.jsonl	6807	后续生成式评估
音频	data/audio/.../*.mp3	随 JSON 引用	模型主要输入

二分类样本长什么样

每条样本大致包含三类信息：

- prompt：系统提示和用户提示，用户提示里包含音频引用和文本说明。
- audio_url：相对音频路径，例如 audio/POS-imitate-4/tts_test1139/tts_test1139.mp3。
- answer：真实标签，normal 或 fraud。代码只相信 answer，不从文件夹名猜标签。

```
{
  "prompt": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": [
      {"type": "audio", "audio_url": "audio/.../call.mp3"},
      {"type": "text", "text": "■■■■■■■■"}
    ]
  }
],
```

```
"answer": "fraud"
}
```

一个容易踩的点：文本字段更像任务提示，不一定是通话转写。当前框架保留文本分支，是为了后续可以把 ASR 转写文本接进去。

3. 模型架构

当前模型可以理解为一个很清楚的流水线：

阶段	输入	处理模块	输出
音频编码	MP3 电话音频	Whisper encoder	audio embedding
文本编码	prompt 或 ASR 文本	中文 RoBERTa/BERT	text embedding
特征融合	两个 embedding	torch.cat 拼接	融合向量
分类	融合向量	LayerNorm + Linear + GELU + Dropout + Linear	normal/fraud logits

核心代码结构

AudioTextFusionClassifier 的输入维度是 audio_dim + text_dim。forward 时先把两个特征拼接，再经过小型神经网络得到 2 个 logits。训练时使用 cross entropy loss。

```
features = torch.cat([audio_features.float(), text_features.float()], dim=-1)
logits = self.classifier(features)
loss = F.cross_entropy(logits, labels.long())
```

为什么要缓存 embedding

- Whisper 和 RoBERTa 参数量大，反复参与训练会慢且吃显存。
- 缓存后，训练阶段只读 .pt 特征文件，速度快很多。
- 调 MLP 的学习率、隐藏层、dropout 时，不需要重新跑音频编码。

4. 代码文件地图

文件	作用	新手阅读顺序
src/teledeceit/data.py	读取 JSON，生成 BinarySample，映射标签	1
src/teledeceit/features.py	读取缓存 .pt，提供 Dataset	2
src/teledeceit/model.py	定义音频+文本融合分类器	3
src/teledeceit/training.py	训练一轮、评估、移动 batch 到 GPU	4
src/teledeceit/metrics.py	计算 accuracy、precision、recall、F1	5
scripts/cache_multimodal_features.py	生成训练/测试特征缓存	6
scripts/train_fusion_classifier.py	训练 MLP 分类头并保存 best_model.pt	7
scripts/predict_binary.py	用 checkpoint 导出预测 CSV	8

测试文件在验证什么

- test_data_pipeline.py: 验证 JSON 解析、标签映射、batch 顺序。
- test_feature_dataset.py: 验证缓存特征文件能按行读取。
- test_model_and_metrics.py: 验证模型输出维度和指标计算。
- test_training_loop.py: 用小型随机特征跑一轮训练和评估。

5. 从零复现步骤

Step 0: 确认环境

```
python -m pip install -r requirements.txt
python -m pip install -e .
nvidia-smi
python -c "import torch; print(torch.cuda.is_available())"
```

Step 1: 先跑测试

```
python -m unittest discover -s tests -v
```

测试通过后再跑训练。这样如果后面出错，你能区分是环境问题、数据问题还是模型训练问题。

Step 2: 先用 limit 做小样本冒烟测试

```
python scripts/cache_multimodal_features.py ^
--json-path data/binary_classification/binary_classification/train.json ^
--data-root data ^
--output artifacts/features/debug_train_32.pt ^
--batch-size 1 ^
--limit 32 ^
--device cuda
```

如果 32 条能跑通，再跑全量。新手不要一开始就跑 4000 条，否则排错会很慢。

Step 3: 缓存全量训练和测试特征

```
python scripts/cache_multimodal_features.py ^
--json-path data/binary_classification/binary_classification/train.json ^
--data-root data ^
--output artifacts/features/binary_train_whisper_small_roberta.pt ^
--audio-model openai/whisper-small ^
--text-model hfl/chinese-roberta-wwm-ext ^
--batch-size 2 ^
--max-audio-seconds 30 ^
--device cuda
```

```
python scripts/cache_multimodal_features.py ^
--json-path data/binary_classification/binary_classification/test.json ^
--data-root data ^
--output artifacts/features/binary_test_whisper_small_roberta.pt ^
--audio-model openai/whisper-small ^
--text-model hfl/chinese-roberta-wwm-ext ^
--batch-size 2 ^
--max-audio-seconds 30 ^
--device cuda
```

Step 4: 训练分类器

```
python scripts/train_fusion_classifier.py ^
--train-cache artifacts/features/binary_train_whisper_small_roberta.pt ^
--test-cache artifacts/features/binary_test_whisper_small_roberta.pt ^
```

```
--output-dir runs/binary_fusion_whisper_small_roberta ^
--epochs 20 ^
--batch-size 64 ^
--hidden-dim 256 ^
--lr 0.001 ^
--device cuda
```

Step 5: 导出预测

```
python scripts/predict_binary.py ^
--checkpoint runs/binary_fusion_whisper_small_roberta/best_model.pt ^
--cache artifacts/features/binary_test_whisper_small_roberta.pt ^
--output-csv runs/binary_fusion_whisper_small_roberta/test_predictions.csv ^
--device cuda
```

6. 当前训练日志摘要

项目中已有 TRAINING_LOG.md，记录了一次完整训练。日志显示实际完成了测试、特征缓存、训练和预测。

环节	结果
单元测试	7/7 通过
训练集特征缓存	4000 行，约 19 分钟，输出约 25 MB
测试集特征缓存	400 行，约 2 分钟，输出约 2.5 MB
最佳 epoch	epoch 16
最佳指标	Accuracy/Precision/Recall/F1 均为 1.0000，TP=200，TN=200，FP=0，FN=0
预测导出	400 条测试预测，日志记录 normal 与 fraud 各 200 条

重要提醒：测试集满分很漂亮，但新手不要把它等同于真实业务可用。可能原因包括数据分布简单、合成音频线索明显、训练/测试生成方式相近等。下一步应做外部数据验证、阈值分析和错误样本复查。

7. 常见问题与排错

问题	现象	解决办法
MP3 加载失败	torchaudio 找不到 backend	安装 FFmpeg；脚本已有 ffmpeg fallback
Whisper 长度报错	expects mel input features length 3000	脚本已 pad/截断到 3000 帧
CUDA OOM	显存不足	batch-size 降到 1；使用 whisper-base；max-audio-seconds 降到 20
HuggingFace 403 警告	safetensors_conversion 后台警告	通常不影响运行，可先忽略
训练很慢	缓存阶段耗时长	先用 --limit 32 检查，再全量；缓存只需做一次

显存友好参数

- 缓存阶段：batch-size=1 或 2。
- 音频模型：先用 openai/whisper-small，OOM 时换 openai/whisper-base。

- 音频长度: `max-audio-seconds=30`, OOM 时改为 20。
- 分类训练: `batch-size=64` 通常很轻, 因为只训练 MLP。

8. 新手学习路线

第一阶段: 能跑起来

- 照第 5 章跑通 `limit=32` 的特征缓存。
- 查看 `.pt` 文件大小, 理解缓存特征是训练输入。
- 跑 1 个 epoch 的训练, 观察 `metrics.json`。

第二阶段: 理解每个数字

- 阅读 `metrics.py`, 手算一个小例子的 TP、FP、TN、FN。
- 阅读 `model.py`, 画出输入维度如何拼接。
- 修改 `hidden-dim` 或 `dropout`, 观察 F1 是否变化。

第三阶段: 做一个有价值的改进

- 用 Whisper 生成 ASR 文本, 替换当前 `prompt text`。
- 增加阈值调节: 例如 `fraud_probability > 0.4` 就判 `fraud`。
- 抽查错误样本, 写出误判原因。

第四阶段: 再考虑 SFT/LoRA

等你能解释当前二分类模型的每一步, 再用 SFT 数据做音频问答或诈骗理由生成。那时需要学习 LoRA、量化加载、梯度累积和生成式评估。

9. 术语表

术语	解释
embedding	把音频或文本压缩成一串数字向量, 供模型计算。
encoder	编码器, 把原始输入变成 embedding 的模型部分。
MLP	多层感知机, 本项目里是很小的分类头。
logits	softmax 前的原始分类分数。
loss	模型预测和真实标签之间的差距, 训练时要让它变小。
precision	判为 fraud 的样本里有多少是真的 fraud。
recall	所有 fraud 样本里有多少被找出来。电诈检测通常很看重它。
F1	precision 和 recall 的综合指标。
checkpoint	保存下来的模型权重文件, 例如 <code>best_model.pt</code> 。
LoRA	一种低显存微调大模型的方法, 适合进阶阶段。

10. 复现检查清单

- 能运行 `python -m unittest discover -s tests -v`。
- 能用 `--limit 32` 生成 debug 特征缓存。
- 全量 train/test 特征缓存都存在。
- `runs/.../best_model.pt` 存在。
- `runs/.../metrics.json` 能看到每个 epoch 的指标。
- `test_predictions.csv` 中有 `sample_id`、`prediction`、`fraud_probability` 三列。
- 如果改了数据或模型，重新运行测试，并保存新的训练日志。

提示 你可以把这份 PDF 当作路线图。真正学习时，不要只跑命令；每跑完一步，都打开对应脚本，确认输入是什么、输出是什么、文件保存在哪里。